

Linux Server Hardening & Secure Configuration

Final Project Report

Project Started: February 10, 2026
Last Updated: August 28, 2026
Status: Completed (Phase 4 of 4)

1. Executive Summary



The **Linux Server Hardening & Secure Configuration** project has been successfully transformed from a static documentation repository into a fully automated, idempotent, and production-ready DevSecOps framework. This framework systematically applies security baseline controls across critical OS subsystems while ensuring strict administrative safety, automated configuration backups, and built-in diagnostic auditing.

2. Architecture & Design Principles

The framework was built with safety as the primary directive. It orchestrates hardening across multiple modules using the following mechanisms:

- **Idempotency:** Modules can be run repeatedly without duplicating configurations or corrupting files.
- **Atomic Backups:** Before any configuration (`/etc/ssh`, `/etc/ufw`, etc.) is modified, a timestamped `.tar.gz` backup is taken, ensuring quick rollback.
- **Validation-First Deployment:** Changes to critical services (e.g., SSH) are strictly validated using `sshd -t` before the daemon is reloaded.
- **Dry-Run Mode:** Operators can simulate hardening (`--dry-run`) to preview the exact configuration changes without altering the host.

3. Hardening Phases & Applied Controls

Phase 1: Core Automation & Safety Foundation

Established the central framework (`harden.sh`), logging library, and backup engine. Implemented system environment checks to enforce root privilege execution and strictly supported operating systems (Ubuntu, Debian, Kali Linux).

Phase 2: Access & Perimeter Security

- **OpenSSH Server (`02-ssh.sh`):** Enforced key-based authentication (`PubkeyAuthentication yes`), disabled root login, and denied empty passwords. Safely disabled password authentication only after successfully verifying the existence of `authorized_keys`.


```
kali@kali: ~/Desktop/hardening-project

Session Actions Edit View Help

[+] Initializing program
- Detecting OS... [ DONE ]
- Checking profiles... [ DONE ]

Program version: 3.1.6
Operating system: Linux
Operating system name: Kali Linux
Operating system version: Rolling release
End-of-life: UNKNOWN
Kernel version: 6.18.5-kali
Hardware platform: x86_64
Hostname: kali

Profiles:
Log file: /etc/lynis/default.prfl
Report file: /var/log/lynis.log
Report version: 1.0
Plugin directory: /etc/lynis/plugins

Auditor: [Not Specified]
Language: en
Test category: all
Test group: all

- Program update status... [ NO UPDATE ]

[+] System tools
- Scanning available tools...
- Checking system binaries...

[+] Plugins (phase 1)
Note: plugins have more extensive tests and may take several minutes to complete

- Plugin: debian
[+] Debian Tests
- Checking for system binaries that are required by Debian Tests...
- Checking /bin... [ FOUND ]
- Checking /sbin... [ FOUND ]
- Checking /usr/bin... [ FOUND ]
- Checking /usr/sbin... [ FOUND ]
- Checking /usr/local/bin... [ FOUND ]
- Checking /usr/local/sbin... [ FOUND ]
- Authentication:
- PAM (Pluggable Authentication Modules):
[WARNING]: Test DEB-0001 had a long execution: 34.217922 seconds
- libpam-tmpdir [ Not Installed ]
- File System Checks:
- DM-Crypt, Cryptsetup & Cryptmount:
[WARNING]: Test DEB-0280 had a long execution: 15.737772 seconds
- Software:
- apt-listbugs [ Not Installed ]
- apt-listchanges [ Not Installed ]
- needrestart [ Not Installed ]
- fail2ban [ Not Installed ]
]

[+] Boot and services
- Service Manager [ systemd ]
- Checking UEFI boot [ DISABLED ]
- Checking presence GRUB2 [ FOUND ]
- Checking for password protection [ NONE ]
- Check running services (systemctl) [ DONE ]
- Result: found 23 running services
- Check enabled services at boot (systemctl) [ DONE ]
```

4. Testing & Validation

All modules were validated against rigorous criteria:

1. **ShellCheck Compliance:** Adherence to standard Bash best practices, including safe variable quoting and `set -Euo pipefail`.
2. **Environment Isolation:** Testing verified that the framework correctly aborted in non-Linux or unsupported OS environments without damage.
3. **Execution Confidence:** Full execution (Phase 1-4) on target VMs successfully applied configurations, activated firewalls, and retained administrative access.

5. Conclusion

The Linux Server Security Hardening Framework is now robust, complete, and production-ready. It provides an essential automated toolchain for DevSecOps engineers to quickly deploy, verify, and document a secure baseline across server fleets.